mini_project _correlation_regression

October 28, 2026

[9]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[3]:
```
# 1. Load dataset
df = pd.read_csv("smart_manufacturing_data.csv")
```

[4]:
```
df
```

[4]:

	timestamp	machine_id	temperature	vibration	humidity	pressure	energy_consumption	machine_status	anomaly_flag	predicted_remaining_life	failure_type	downtime_risk
0	2025-01-01 00:00:00	39	78.61	28.65	79.96	3.73	2.16	1	0			
1	2025-01-01 00:01:00	29	68.19	57.28	35.94	3.64	0.69	1	0			
2	2025-01-01 00:02:00	15	98.94	50.20	72.06	1.00	2.49	1	1			
3	2025-01-01 00:03:00	43	90.91	37.65	30.34	3.15	4.96	1	1			
4	2025-01-01 00:04:00	8	72.32	40.69	56.71	2.68	0.63	2	0			
…	…	…	…	…	…	…	…	…	…			
99995	2025-03-11 10:35:00	3	71.47	54.34	46.72	1.83	3.05	1	0			
99996	2025-03-11 10:36:00	23	87.23	34.99	33.84	4.67	4.47	1	0			
99997	2025-03-11 10:37:00	10	93.01	51.79	64.28	4.04	3.62	1	1			
99998	2025-03-11 10:38:00	47	84.41	22.35	35.18	2.88	2.37	1	0			
99999	2025-03-11 10:39:00	32	73.53	67.09	74.35	2.15	2.50	1	0			

```
0                            106           Normal            0.0
1                            320           Normal            0.0
2                             19           Normal            1.0
3                             10           Normal            1.0
4                             65  Vibration Issue            0.0
…                            …                …              …
99995                        313           Normal            0.0
99996                        439           Normal            0.0
99997                         30           Normal            1.0
99998                        487           Normal            0.0
99999                        446           Normal            0.0

       maintenance_required
0                         0
1                         0
2                         1
3                         1
4                         1
…                       …
99995                     0
99996                     0
99997                     1
99998                     0
99999                     0

[100000 rows x 13 columns]
```

[10]:
```python
columns = [
    "temperature",
    "vibration",
    "humidity",
    "pressure",
    "energy_consumption",
    "predicted_remaining_life"
]
data = df[columns].apply(pd.to_numeric, errors="coerce")
data = data.replace([np.inf, -np.inf], np.nan).dropna()

print("Number of rows used:", len(data))

# ========================================
# STEP 2: CORRELATION MATRIX
# ========================================

corr = data.corr()

print("\nCorrelation Matrix:")
```

```python
print(corr.round(3))

# ========================================
# STEP 3: CORRELATION HEATMAP
# ========================================

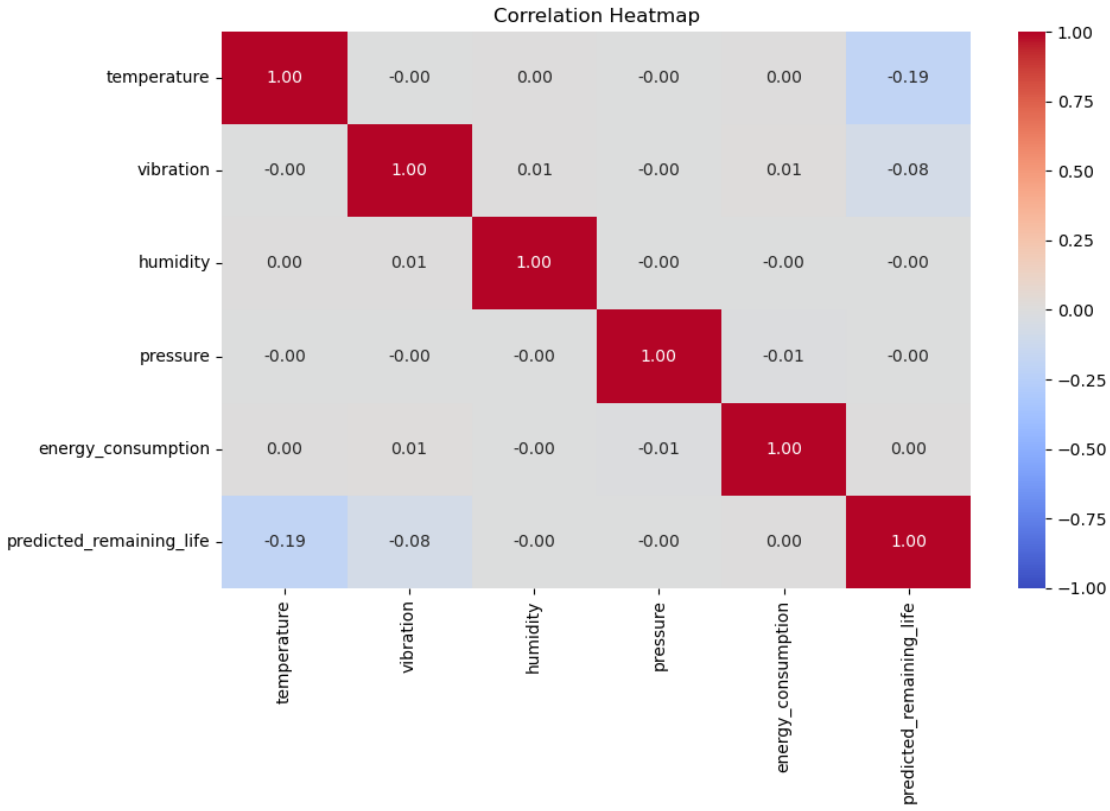
plt.figure(figsize=(10, 7))
sns.heatmap(
    corr, annot=True, cmap="coolwarm",
    vmin=-1, vmax=1, fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

```
Number of rows used: 100000

Correlation Matrix:
                          temperature  vibration  humidity  pressure  \
temperature                     1.000     -0.003     0.003    -0.001
vibration                      -0.003      1.000     0.006    -0.000
humidity                        0.003      0.006     1.000    -0.000
pressure                       -0.001     -0.000    -0.000     1.000
energy_consumption              0.003      0.008    -0.003    -0.008
predicted_remaining_life       -0.193     -0.076    -0.001    -0.002

                          energy_consumption  predicted_remaining_life
temperature                            0.003                    -0.193
vibration                              0.008                    -0.076
humidity                              -0.003                    -0.001
pressure                              -0.008                    -0.002
energy_consumption                     1.000                     0.005
predicted_remaining_life               0.005                     1.000
```



[11]:
```python
# ========================================
# STEP 4: STRONGEST CORRELATIONS
# ========================================

upper = corr.where(
    np.triu(np.ones(corr.shape), k=1).astype(bool)
)

pairs = upper.stack().sort_values()

print("\nStrongest Negative Correlations:")
print(pairs.head(5).round(3))

print("\nStrongest Positive Correlations:")
print(pairs.tail(5).sort_values(ascending=False).round(3))

print("\nCorrelation with Remaining Life:")
print(
    corr["predicted_remaining_life"]
    .drop("predicted_remaining_life")
    .sort_values()
```

```
    .round(3)
)
```

```
Strongest Negative Correlations:
temperature  predicted_remaining_life   -0.193
vibration    predicted_remaining_life   -0.076
pressure     energy_consumption         -0.008
temperature  vibration                  -0.003
humidity     energy_consumption         -0.003
dtype: float64

Strongest Positive Correlations:
vibration           energy_consumption          0.008
                    humidity                    0.006
energy_consumption  predicted_remaining_life    0.005
temperature         energy_consumption          0.003
                    humidity                    0.003
dtype: float64

Correlation with Remaining Life:
temperature          -0.193
vibration            -0.076
pressure             -0.002
humidity             -0.001
energy_consumption    0.005
Name: predicted_remaining_life, dtype: float64
```

[12]:
```
# ========================================
# STEP 5: SCATTER PLOTS
# ========================================

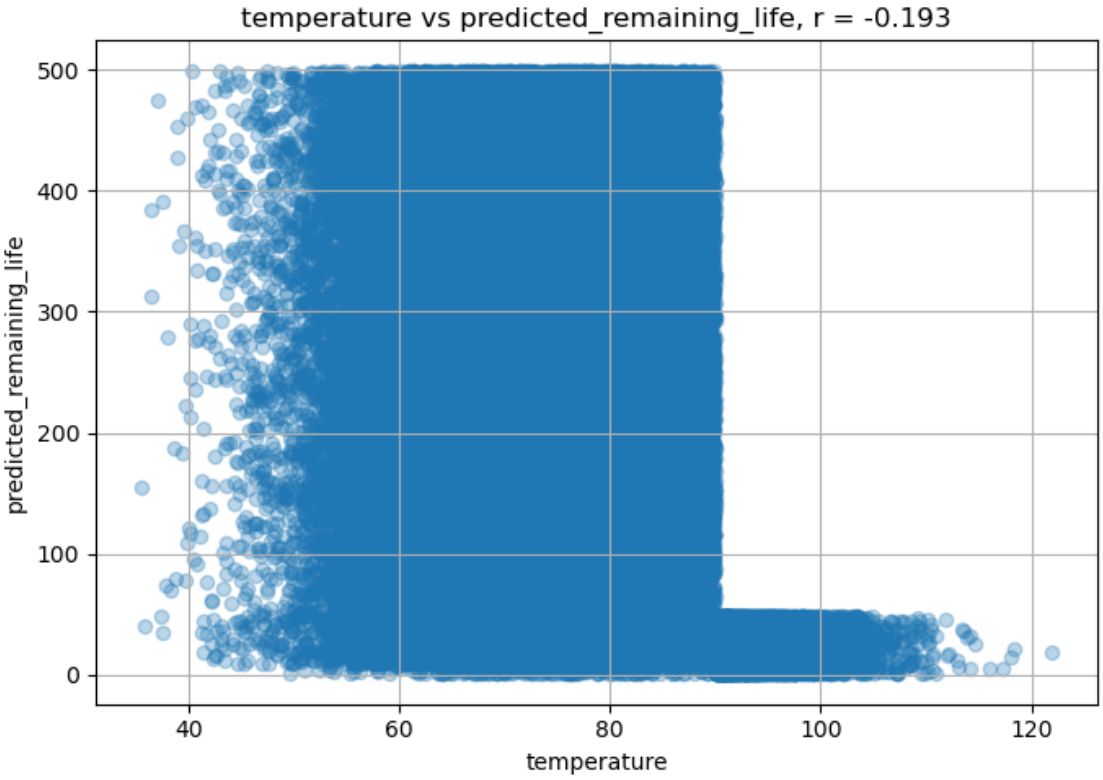
plot_pairs = [
    ("temperature", "predicted_remaining_life"),
    ("temperature", "energy_consumption"),
    ("vibration", "predicted_remaining_life")
]

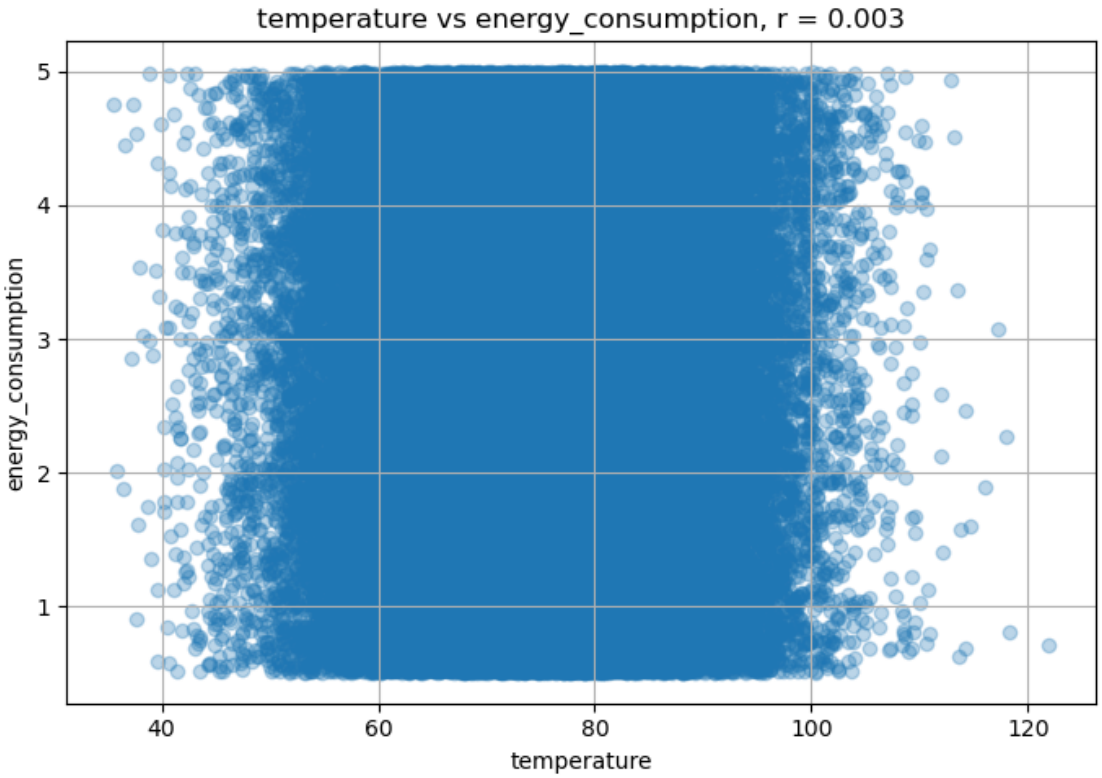
for x_col, y_col in plot_pairs:
    r = data[x_col].corr(data[y_col])

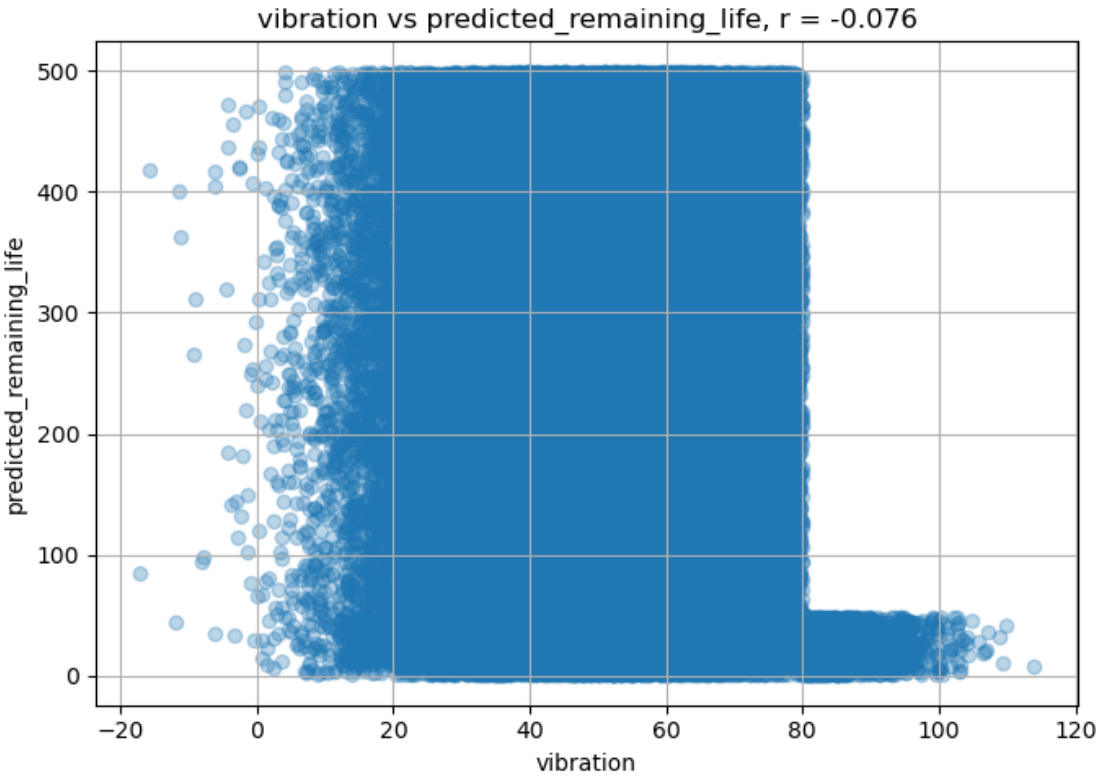
    plt.figure(figsize=(7, 5))
    plt.scatter(data[x_col], data[y_col], alpha=0.3)

    plt.xlabel(x_col)
    plt.ylabel(y_col)
    plt.title(f"{x_col} vs {y_col}, r = {r:.3f}")
```

```
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```




temperature vs energy_consumption, r = 0.003
energy_consumption
temperature
5
4
3
2
1
40
60
80
100
120


vibration vs predicted_remaining_life, r = -0.076
predicted_remaining_life
vibration
500
400
300
200
100
0
−20
0
20
40
60
80
100
120

[13]:
```
# ========================================
# STEP 6: SIMPLE LINEAR REGRESSION
# X = Temperature
# Y = Predicted Remaining Life
# ========================================

X = data["temperature"].to_numpy()
Y = data["predicted_remaining_life"].to_numpy()

# 80% training data and 20% testing data
np.random.seed(42)
indices = np.random.permutation(len(X))

split = int(0.8 * len(X))
train = indices[:split]
test = indices[split:]

X_train, Y_train = X[train], Y[train]
X_test, Y_test = X[test], Y[test]

# Calculate slope and intercept manually
m = np.sum((X_train - X_train.mean()) *
           (Y_train - Y_train.mean())) / np.sum(
           (X_train - X_train.mean()) ** 2)

c = Y_train.mean() - m * X_train.mean()

print("\nRegression Equation:")
print(f"Y = {m:.4f} * X + {c:.4f}")
```

```
Regression Equation:
Y = -2.8960 * X + 451.4713
```

[16]:
```
# ========================================
# STEP 7: PREDICTIONS
# ========================================

Y_pred = m * X_test + c

results = pd.DataFrame({
    "Temperature": X_test[:5],
    "Actual Life": Y_test[:5],
    "Predicted Life": Y_pred[:5],
    "Error": Y_test[:5] - Y_pred[:5]
```

```
})

print("\nFirst 5 Test Predictions:")
print(results.round(3).to_string(index=False))
```

```
First 5 Test Predictions:
 Temperature  Actual Life  Predicted Life   Error
       72.81          460         240.610 219.390
       80.88          400         217.239 182.761
       60.79          494         275.421 218.579
       77.28          332         227.665 104.335
       68.35          207         253.527 -46.527
```

[17]:
```
# ========================================
# STEP 8: MODEL EVALUATION
# ========================================

errors = Y_test - Y_pred

MAE = np.mean(np.abs(errors))
MSE = np.mean(errors ** 2)
RMSE = np.sqrt(MSE)

SS_res = np.sum(errors ** 2)
SS_total = np.sum((Y_test - Y_test.mean()) ** 2)
R2 = 1 - SS_res / SS_total if SS_total != 0 else np.nan

print("\nRegression Evaluation:")
print("MAE:", round(MAE, 3))
print("MSE:", round(MSE, 3))
print("RMSE:", round(RMSE, 3))
print("R-squared:", round(R2, 4))
```

```
Regression Evaluation:
MAE: 128.895
MSE: 21858.537
RMSE: 147.846
R-squared: 0.0353
```

[18]:
```
# ========================================
# STEP 9: REGRESSION LINE
# ========================================

order = np.argsort(X_test)
```

```
plt.figure(figsize=(8, 5))
plt.scatter(X_test, Y_test, alpha=0.3, label="Actual values")
plt.plot(X_test[order], Y_pred[order],
         color="red", label="Regression line")

plt.xlabel("Temperature")
plt.ylabel("Predicted Remaining Life")
plt.title("Simple Linear Regression")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

